ITM SKILLS UNIVERSITY

School of Future Tech

Case Study Report

on

Telemedicine Backend System Using C++

By

Ved Patil

Roll No - 150096725005

1. Introduction to the Case Study

Healthcare services are increasingly adopting digital technologies to improve accessibility and efficiency. Telemedicine allows patients to consult doctors remotely and enables healthcare providers to manage consultations electronically.

This case study focuses on designing and implementing a **Telemedicine Backend System using C++**. The system simulates a simplified backend used in telemedicine platforms to manage doctors, patients, and consultations.

The system allows users to register doctors and patients, schedule consultations, and store consultation records permanently using file handling. The objective is to demonstrate how object-oriented programming concepts such as **classes, inheritance, polymorphism, vectors, exception handling, and file handling** can be used to build a simple healthcare management backend.

This project provides a structured approach to storing and managing telemedicine data using a command-line interface.

2. Problem Statement / Case Background (Abstract)

Background

Traditional hospital record systems often rely on manual record keeping, which can lead to inefficiencies, data loss, and difficulty in retrieving information. Telemedicine platforms require a digital system to store and manage patient consultations effectively.

A backend system is required to manage the following tasks:

- Registering doctors and patients
- Scheduling consultations
- Recording symptoms, diagnosis, and treatment status
- Storing consultation records permanently

Abstract

This case study presents the development of a **Telemedicine Backend System implemented in C++**. The system uses object-oriented programming principles to model entities such as doctors, patients, and appointments.

Doctors and patients can be registered through a command-line interface. Consultations can then be scheduled by linking doctor IDs and patient IDs. Consultation details such as symptoms, diagnosis, and treatment status are stored using **C++ file handling** in a text file named **records.txt**.

The system also incorporates **exception handling** to prevent duplicate IDs and invalid operations. This ensures that the system maintains data integrity and reliability. The implementation demonstrates how core C++ programming concepts can be used to simulate a simple telemedicine backend system.

3. Case Study Design

The Telemedicine Backend System is designed as a simple menu-driven application with the following components:

1. Doctor Registration

Users can register doctors by providing:

- Doctor ID

- Name
- Specialization

The system verifies that the doctor ID is unique before storing the doctor in the system.

2. Patient Registration

Users can register patients by providing:

- Patient ID
- Name
- Age

The system checks if the patient ID already exists to avoid duplication.

3. Consultation Scheduling

Once doctors and patients are registered, a consultation can be scheduled by entering:

- Doctor ID
- Patient ID
- Date
- Symptoms
- Diagnosis

- Treatment status

The system validates whether both doctor and patient exist before creating the appointment.

4. Data Storage

All consultation details are stored permanently in a file called **records.txt** using file handling.

5. Menu Driven Interface

The system provides a simple command-line menu where users can choose operations such as registering doctors, registering patients, scheduling consultations, or exiting the system.

4. Methods & Algorithms / Technology Applied

Object-Oriented Programming (OOP)

The system is designed using OOP principles including:

- **Encapsulation**
  Classes are used to encapsulate data and behavior.

- **Inheritance**
  The `Doctor` and `Patient` classes inherit from a base class called `Person`.

- **Polymorphism**
  The `display()` function is declared as a virtual function in the `Person` class and implemented differently in derived classes.

Data Structures

The program uses **vectors** from the C++ Standard Library to store doctors and patients dynamically during execution.

File Handling

Consultation records are stored using "**ofstream"** in a file called `records.txt`.
This ensures that the consultation data is preserved even after the program exits.

Exception Handling

Exception handling is used to manage errors such as:

- Duplicate doctor IDs
- Duplicate patient IDs
- Invalid doctor or patient IDs
- Invalid menu choices

This improves the reliability and robustness of the system.

Technologies Used

Programming Language:

- C++

Libraries Used:

- iostream (input/output operations)
- vector (dynamic data storage)

- fstream (file handling)
- stdexcept (exception handling)

Development Environment:

- VS Code / any C++ compiler

Data Storage:

- Text file (`records.txt`)

5. Implementation Details and Snapshots

The implementation of the Telemedicine Backend System consists of multiple classes that represent real-world entities.

1. Person Class

The `Person` class acts as the base class and contains common attributes such as:

- ID
- Name

```cpp
// ================= PERSON CLASS =================
class Person {
protected:
    int id;
    string name;

public:
    Person(int id, string name) {
        this->id = id;
        this->name = name;
    }

    int getId() const { return id; }

    virtual void display() const = 0;
};
```

It also contains a virtual function `display()` which is implemented by derived classes.

2. Doctor Class

The `Doctor` class inherits from `Person` and includes an additional attribute:

- Specialization

```
// ================= DOCTOR CLASS =================
class Doctor : public Person {
private:
    string specialization;

public:
    Doctor(int id, string name, string specialization)
        : Person(id, name), specialization(specialization) {}

    void display() const override {
        cout << "Doctor ID: " << id
             << " | Name: " << name
             << " | Specialization: " << specialization << endl;
    }
};
```

This class represents doctors registered in the system.

3. Patient Class

The `Patient` class also inherits from `Person` and contains:

- Age

```cpp
// ================= PATIENT CLASS =================
class Patient : public Person {
private:
    int age;

public:
    Patient(int id, string name, int age)
        : Person(id, name), age(age) {}

    void display() const override {
        cout << "Patient ID: " << id
             << " | Name: " << name
             << " | Age: " << age << endl;
    }
```

This class stores information about patients.

4. Appointment Class

The `Appointment` class stores consultation information such as:

- Doctor ID
- Patient ID
- Date
- Symptoms
- Diagnosis
- Status

```cpp
// ================= APPOINTMENT CLASS =================
class Appointment {
private:
    int doctorId;
    int patientId;
    string date;
    string symptoms;
    string diagnosis;
    string status;

public:
    Appointment(int dId, int pId, string date,
                string symptoms, string diagnosis, string status)
        : doctorId(dId), patientId(pId),
          date(date), symptoms(symptoms),
          diagnosis(diagnosis), status(status) {}

    void saveToFile() {
        ofstream file("records.txt", ios::app);

        if (!file)
            throw runtime_error("Error opening records.txt");

        file << "Doctor ID: " << doctorId
             << " | Patient ID: " << patientId
             << " | Date: " << date
             << " | Symptoms: " << symptoms
             << " | Diagnosis: " << diagnosis
             << " | Status: " << status << endl;

        file.close();
```

The class also contains a function that writes the consultation details to **records.txt**.

5. Helper Functions

Two helper functions are implemented:

- `doctorExists()` → Checks if a doctor ID already exists.
- `patientExists()` → Checks if a patient ID already exists.

```
//  HELPER FUNCTIONS
bool doctorExists(const vector<Doctor>& doctors, int id) {
    for (const auto& d : doctors)
        if (d.getId() == id)
            return true;
    return false;
}

bool patientExists(const vector<Patient>& patients, int id) {
    for (const auto& p : patients)
        if (p.getId() == id)
            return true;
    return false;
}
```

These functions help prevent duplicate entries.

6. Results and Conclusion

Results

The Telemedicine Backend System successfully demonstrates how a healthcare consultation management system can be implemented using C++.

Key achievements of the system include:

- Efficient registration of doctors and patients
- Secure scheduling of consultations
- Permanent storage of consultation records
- Prevention of duplicate entries using validation
- Robust error handling through exception management

The system provides a structured way of managing telemedicine consultations using simple programming techniques.

Conclusion

This case study demonstrates the implementation of a **Telemedicine Backend System using C++** that manages doctors, patients, and consultations. The project highlights the use of object-oriented programming concepts, dynamic data structures, and file handling to create a functional backend simulation.

Although the system is implemented as a command-line application, the same concepts can be extended to build more advanced systems such as web-based telemedicine platforms or hospital management systems.

This project provides a strong foundation for understanding how backend systems in healthcare applications can be designed and implemented.

7. References

- C++ Documentation – Standard Library
- C++ File Handling Documentation
- Object-Oriented Programming Concepts in C++
- Online tutorials and documentation related to C++ vectors and exception handling
- Telemedicine and healthcare management system research articles

8. Limitations and Future Improvements

Although the Telemedicine Backend System works successfully, there are several improvements that can be made in the future.

- **Graphical User Interface (GUI):** The system currently uses a command-line interface. A graphical interface could make it more user-friendly.

- **Database Integration:** Instead of storing data in a text file, a database such as MySQL or SQLite could be used for better data management.

- **Record Viewing Feature:** The system currently stores consultation data but does not allow users to view or search past records.

- **User Authentication:** A login system for doctors and administrators could be added to improve security.

- **Advanced Appointment Scheduling:** Future versions could include time-slot scheduling and prevent double bookings.

These improvements would make the system more practical and suitable for real-world telemedicine applications.